

Manual Tecnico CleanHome

Guia de implementacion, configuracion, ejecucion, integracion y mantenimiento para desarrolladores.

Fecha de actualizacion: 23 de mayo de 2026.

Version del documento: 2.0.

Repositorio: CleanHome.

1. Objetivo del manual

Este manual tecnico tiene como objetivo permitir que un desarrollador pueda levantar CleanHome desde cero, entender su arquitectura, configurar sus conexiones, conocer sus archivos principales, consumir sus endpoints, probar la integracion y mantener el sistema.

El punto de partida de este manual es un ambiente limpio donde la base de datos no existe. Por eso, el primer flujo recomendado es crear la base MySQL desde cero usando el script oficial del proyecto.

El documento cubre backend, base de datos, frontend web administrativo, aplicacion Android, SQLite local, endpoints, configuraciones, pruebas, seguridad, troubleshooting, glosario y conclusiones.

2. Alcance tecnico

- * Backend REST construido con Node.js, Express y MySQL.
- * Base central MySQL creada desde cleanhome-backend/database/cleanhome.sql.
- * Panel web administrativo servido desde cleanhome-admin-frontend.
- * Aplicacion Android creada en Kotlin con vistas XML.
- * Consumo de API desde Android mediante Retrofit y Gson.
- * Almacenamiento local Android mediante SQLiteOpenHelper.
- * Autenticacion mediante JWT.
- * Roles Cliente y Admin.
- * Gestion de servicios, solicitudes, estados y personal.
- * Preparacion para un despliegue futuro en servidor real.

3. Requisitos previos

- * Git instalado.
- * Node.js y npm instalados.
- * MySQL Server instalado y en ejecucion.
- * Android Studio instalado.
- * Emulador Android o dispositivo fisico.
- * Navegador web moderno.
- * Acceso a terminal PowerShell o equivalente.
- * Puerto 3000 libre para el backend.

* Puerto 5173 libre para el frontend web.

4. Arquitectura general del sistema

CleanHome esta dividido en tres capas de aplicacion y dos capas de persistencia.

Aplicacion Android -> Backend REST Express -> MySQL

Panel Web Admin -> Backend REST Express -> MySQL

Aplicacion Android -> SQLite local

El backend es la unica capa que accede directamente a MySQL. El panel web y la app Android consumen la API REST. SQLite se usa en Android como almacenamiento local para sesion, catalogo, historial y soporte de sincronizacion.

5. Estructura del repositorio

```
CleanHome/
|- README.md
|- .gitignore
|- cleanhome-backend/
|- cleanhome-admin-frontend/
|- cleanHome-androidApp/
|- documentacion/
|- cdocumentacion/
`- tools/
```

- * README.md: guia general del proyecto y rutas principales.
- * .gitignore: excluye archivos locales o generados como .env, node_modules, build, .gradle y local.properties.
- * cleanhome-backend: API REST, base de datos, rutas, controladores y middlewares.
- * cleanhome-admin-frontend: panel web administrativo.
- * cleanHome-androidApp: aplicacion Android cliente/admin.
- * documentacion: PDF y fuente Markdown del manual tecnico.
- * cdocumentacion: manual tecnico Word historico.
- * tools: scripts auxiliares para generar documentacion.

6. Estructura del backend

```
cleanhome-backend/
|- README.md
|- package.json
|- package-lock.json
|- database/
|   |- cleanhome.sql
|   |- sqlite_android_cleanhome.sql
|   `-- migrations/
```

```
|      `-- 001_sync_support.sql
`-- src/
    |-- app.js
    |-- server.js
    |-- config/
    |   |-- db.js
    |-- controllers/
    |   |-- auth.controller.js
    |   |-- servicios.controller.js
    |   |-- solicitudes.controller.js
    |   |-- admin.controller.js
    |-- middlewares/
    |   |-- auth.middleware.js
    |   |-- role.middleware.js
    |-- routes/
    |   |-- auth.routes.js
    |   |-- servicios.routes.js
    |   |-- solicitudes.routes.js
    |   |-- admin.routes.js
```

- * package.json: define dependencias y scripts npm del backend.
- * src/server.js: punto de arranque; carga dotenv y levanta el servidor.
- * src/app.js: configura Express, CORS, JSON, rutas y manejadores de error.
- * src/config/db.js: configura el pool MySQL usando variables de entorno.
- * controllers: contienen la logica de negocio por modulo.
- * routes: exponen endpoints HTTP y conectan cada ruta con su controlador.
- * middlewares: validan JWT y roles.

7. Creacion de base de datos desde cero

El manual parte de que la base no existe. El flujo recomendado es crearla limpia con el script oficial.

Entrar a MySQL:

```
mysql -u TU_USUARIO -p
```

Crear base vacia:

```
DROP DATABASE IF EXISTS CleanHomeDB;
CREATE DATABASE CleanHomeDB CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
exit
```

Importar esquema y datos semilla:

```
mysql -u TU_USUARIO -p CleanHomeDB < cleanhome-backend/database/cleanhome.sql
```

Si el script ya contiene CREATE DATABASE y USE, tambien puede ejecutarse directamente:

```
mysql -u TU_USUARIO -p < cleanhome-backend/database/cleanhome.sql
```

La migracion 001_sync_support.sql solo se usa si existe una base antigua que se desea actualizar sin borrar datos. En instalaciones desde cero no debe ser el primer paso.

8. Diccionario de base de datos MySQL

- * roles: define los roles del sistema. Campos principales: id_rol, nombre_rol.
- * usuarios: almacena clientes y administradores. Campos principales: id_usuario, nombre, correo, telefono, direccion, password_hash, id_rol, fecha_creacion, updated_at.
- * servicios: catalogo de servicios. Campos principales: id_servicio, nombre, descripcion, duracion_estimada, precio, activo, fecha_creacion, updated_at.
- * personal_limpieza: personal disponible para asignaciones. Campos principales: id_personal, nombre, telefono, disponible, fecha_creacion, updated_at.
- * solicitudes: solicitudes de limpieza. Campos principales: id_solicitud, id_usuario, id_servicio, id_personal, fecha_servicio, hora_servicio, direccion_atencion, estado, client_temp_id, fecha_creacion, updated_at.

Relaciones principales:

- * usuarios.id_rol referencia roles.id_rol.
- * solicitudes.id_usuario referencia usuarios.id_usuario.
- * solicitudes.id_servicio referencia servicios.id_servicio.
- * solicitudes.id_personal referencia personal_limpieza.id_personal.

9. Configuracion del backend

Crear archivo local:

```
cleanhome-backend/.env
```

Variables necesarias:

```
PORT=3000
DB_HOST=localhost
DB_USER=TU_USUARIO
DB_PASSWORD=TU_PASSWORD
DB_NAME=CleanHomeDB
JWT_SECRET=CAMBIAR_EN_CADA_AMBIENTE
JWT_EXPIRES_IN=8h
```

Archivo que lee estas variables:

```
cleanhome-backend/src/config/db.js
```

Archivo que levanta el servidor:

```
cleanhome-backend/src/server.js
```

Nunca subir .env al repositorio. Cada desarrollador debe tener su propia configuracion local.

10. Levantar backend

```
cd cleanhome-backend
npm install
npm start
```

URL esperada:

```
http://localhost:3000
```

Validacion:

```
http://localhost:3000/api/servicios
```

La respuesta esperada es un JSON con la propiedad servicios.

11. Catalogo completo de endpoints

Formato: Metodo | Endpoint | Auth | Rol | Uso.

POST	/api/auth/register	No	Publico	Registrar cliente
POST	/api/auth/login	No	Publico	Iniciar sesion y obtener JWT
GET	/api/servicios	No	Publico	Listar servicios activos
GET	/api/servicios/:id	No	Publico	Obtener detalle de servicio
POST	/api/solicitudes	Si	Cliente	Crear solicitud de servicio
GET	/api/solicitudes/mis-solicitudes	Si	Cliente	Listar historial propio
GET	/api/solicitudes/mis-solicitudes?updated_since=fecha	Si	Cliente	Sincronizacion incremental
POST	/api/solicitudes/sync	Si	Cliente	Sincronizar solicitudes offline
GET	/api/solicitudes/:id	Si	Cliente	Consultar solicitud propia
GET	/api/admin/solicitudes	Si	Admin	Listar todas las solicitudes
GET	/api/admin/personal	Si	Admin	Listar personal de limpieza
PATCH	/api/admin/solicitudes/:id/estado	Si	Admin	Cambiar estado de solicitud
PATCH	/api/admin/solicitudes/:id/asignar-personal	Si	Admin	Asignar personal a solicitud
GET	/api/admin/servicios	Si	Admin	Listar servicios activos e inactivos
POST	/api/admin/servicios	Si	Admin	Crear servicio
PUT	/api/admin/servicios/:id	Si	Admin	Editar servicio

`DELETE /api/admin/servicios/:id`

Si Admin

Desactivar servicio

12. Detalle de endpoints de autenticacion

POST /api/auth/register.

Body:

```
{
  "nombre": "Cliente Demo",
  "correo": "cliente@demo.com",
  "telefono": "700000000",
  "direccion": "San Salvador",
  "password": "ClaveSegura123"
}
```

Respuesta exitosa: mensaje e id_usuario.

POST /api/auth/login.

Body:

```
{
  "correo": "cliente@demo.com",
  "password": "ClaveSegura123"
}
```

Respuesta exitosa: message, token y usuario.

13. Detalle de endpoints de servicios

GET /api/servicios devuelve servicios activos para catalogo cliente.

GET /api/servicios?updated_since=2026-05-23T00:00:00 permite sincronizacion incremental.

GET /api/servicios/:id devuelve un servicio activo especifico.

POST /api/admin/servicios crea un servicio. Requiere JWT admin.

Body:

```
{
  "nombre": "Limpieza Profunda",
  "descripcion": "Servicio completo",
  "duracion_estimada": "3 horas",
  "precio": 45.00,
  "activo": true
}
```

PUT /api/admin/servicios/:id actualiza datos.

DELETE /api/admin/servicios/:id desactiva logicamente el servicio.

14. Detalle de endpoints de solicitudes

POST /api/solicitudes crea una solicitud autenticada.

Body:

```
{
  "id_servicio": 1,
  "fecha_servicio": "2026-05-25",
  "hora_servicio": "09:00",
  "direccion_atencion": "San Salvador",
  "client_temp_id": "uuid-generado-por-android"
}
```

GET /api/solicitudes/mis-solicitudes devuelve el historial del usuario autenticado.

POST /api/solicitudes/sync recibe un arreglo de solicitudes creadas offline.

PATCH /api/admin/solicitudes/:id/estado cambia estado. Estados permitidos: Pendiente, Confirmada, En proceso, Completada, Cancelada.

PATCH /api/admin/solicitudes/:id/asignar-personal asigna un id_personal existente.

15. Frontend web administrativo

Ruta:

cleanhome-admin-frontend/

Instalacion y ejecucion:

```
cd cleanhome-admin-frontend
npm install
npm start
```

URL:

http://localhost:5173

Configuracion API Base recomendada:

http://localhost:3000/api

Funcionalidades:

- * Login admin con JWT.
- * Listar solicitudes.
- * Cambiar estado.
- * Asignar personal.
- * Listar servicios.
- * Crear servicios.
- * Editar servicios.
- * Desactivar servicios.

16. Aplicacion Android

Ruta:

cleanHome-androidApp/

Abrir esta carpeta desde Android Studio.

Archivo de configuracion de API:

```
cleanHome-androidApp/app/src/main/java/com/example/proyectopdm2026_gt01_grupo01_limpieza/api/ApiConfig.kt
```

Emulador Android:

```
const val BASE_URL = "http://10.0.2.2:3000/api/"
```

Dispositivo fisico:

```
const val BASE_URL = "http://IP_DE_TU_PC:3000/api/"
```

No subir IPs personales al repositorio. Usar la IP fisica solo localmente.

Permisos Android:

- * INTERNET.
- * ACCESS_NETWORK_STATE.
- * usesCleartextTraffic=true para HTTP local en desarrollo.

17. Funcionalidades Android cliente

- * Registro de cliente.
- * Login de cliente.
- * Guardado de token JWT en SharedPreferences.
- * Cache de usuario, servicios y solicitudes en SQLite.
- * Catalogo de servicios.
- * Seleccion de servicio.
- * Agenda con selector de fecha y hora.
- * Creacion de solicitud.
- * Historial de solicitudes.
- * Colores de estado en historial.
- * Perfil con cierre de sesion y navegacion de regreso.

18. Funcionalidades Android admin

- * Login con usuario de rol Admin.
- * Listado de solicitudes.
- * Cambio de estado.
- * Listado de servicios.
- * Crear servicio.
- * Editar servicio.
- * Desactivar servicio.

La asignacion de personal se mantiene disponible en el panel web administrativo.

19. SQLite local Android

Script de referencia:

```
cleanhome-backend/database/sqlite_android_cleanhome.sql
```

Implementacion real:

```
cleanHome-androidApp/app/src/main/java/.../data/CleanHomeDbHelper.kt
```

Tablas:

- * usuario_sesion: datos del usuario autenticado y token.
- * servicios: cache local de servicios.
- * solicitudes: historial local de solicitudes.
- * solicitudes_pendientes_sync: cola para solicitudes creadas sin conexion.
- * sync_metadata: fecha de ultima sincronizacion por modulo.

Este script no se importa en MySQL. Es solo para SQLite local del dispositivo.

20. Flujo de levantamiento paso a paso

1. Clonar o abrir el repositorio CleanHome.
2. Iniciar MySQL Server.
3. Crear CleanHomeDB desde cero.
4. Importar cleanhome-backend/database/cleanhome.sql.
5. Crear cleanhome-backend/.env.
6. Configurar DB_HOST, DB_USER, DB_PASSWORD, DB_NAME y JWT_SECRET.
7. Ejecutar npm install en cleanhome-backend.
8. Ejecutar npm start en cleanhome-backend.
9. Validar GET http://localhost:3000/api/servicios.
10. Ejecutar npm install en cleanhome-admin-frontend.
11. Ejecutar npm start en cleanhome-admin-frontend.
12. Abrir http://localhost:5173.
13. Iniciar sesion con admin.
14. Crear un servicio desde el panel web.
15. Abrir cleanHome-androidApp en Android Studio.
16. Confirmar BASE_URL segun emulador o dispositivo fisico.
17. Ejecutar app Android.
18. Registrar cliente o iniciar sesion.
19. Confirmar que el catalogo muestra el servicio creado.
20. Agendar solicitud.
21. Ver solicitud en historial Android.

22. Ver solicitud en panel web o Android admin.

23. Cambiar estado y verificar color actualizado en historial.

21. Pruebas de integracion recomendadas

- * Backend responde /api/servicios.
- * Login admin funciona en web.
- * CRUD de servicios funciona desde web.
- * Android cliente ve servicios creados desde web.
- * Android cliente puede crear solicitud.
- * MySQL registra la solicitud.
- * Web admin ve la solicitud.
- * Android admin ve la solicitud.
- * Cambio de estado se refleja en consultas posteriores.
- * Cierre de sesion limpia la sesion local.

22. Seguridad

- * No subir .env.
- * No subir local.properties.
- * No subir IPs personales.
- * Cambiar JWT_SECRET por ambiente.
- * Cambiar credenciales demo antes de produccion.
- * Mantener contraseñas con bcrypt.
- * Usar HTTPS en despliegue real.
- * Restringir CORS en produccion.
- * Evitar publicar bases de datos reales o dumps con usuarios reales.

23. Solucion de problemas

- * MySQL no conecta: revisar DB_HOST, DB_USER, DB_PASSWORD y DB_NAME en .env.
- * Backend no levanta: revisar puerto 3000, dependencias npm y logs.
- * /api/servicios no responde: confirmar que backend este ejecutandose.
- * Login falla: revisar usuario, hash de contraseña, rol y JWT_SECRET.
- * Android no conecta en emulador: usar 10.0.2.2, no localhost.
- * Android no conecta en celular: usar IP local de la PC y revisar firewall.
- * Catalogo vacio: revisar servicios.activo y respuesta de GET /api/servicios.
- * Solicitud no se crea: revisar token JWT y campos obligatorios.
- * Estado no cambia: revisar rol Admin y endpoint PATCH correspondiente.
- * PDF desactualizado: regenerar con tools/build_manual_tecnico_pdf.py.

24. Mantenimiento

- * Documentar nuevos endpoints en este manual y en README.md.

- * Crear migraciones para cambios de base de datos.
- * Evitar modificar directamente bases productivas sin respaldo.
- * Probar backend, web y Android antes de hacer merge a main.
- * Mantener .gitignore actualizado para evitar credenciales o artefactos generados.
- * Usar commits pequenos y descriptivos.

25. Glosario

- * API: interfaz HTTP que permite que web y Android consuman funcionalidades del backend.
- * Backend: servidor Node.js/Express que contiene logica de negocio y acceso a MySQL.
- * JWT: token firmado usado para autenticar usuarios.
- * Rol: nivel de permiso del usuario, por ejemplo Cliente o Admin.
- * MySQL: motor de base de datos central del sistema.
- * SQLite: base local dentro del dispositivo Android.
- * Retrofit: libreria Android para consumir APIs REST.
- * Endpoint: ruta HTTP expuesta por el backend.
- * Middleware: funcion intermedia en Express para validar auth, roles o datos.
- * CRUD: crear, leer, actualizar y eliminar o desactivar registros.
- * Sincronizacion: proceso de mantener datos locales y remotos consistentes.
- * client_temp_id: identificador local generado por Android para evitar duplicados al sincronizar.
- * .env: archivo local con variables sensibles de configuracion.
- * Admin: usuario con permisos administrativos.
- * Cliente: usuario que solicita servicios de limpieza.

26. Conclusiones

CleanHome queda estructurado como un sistema modular con separacion clara entre API, frontend web, app Android y persistencia. Esta separacion facilita mantenimiento, pruebas y un despliegue futuro.

El backend centraliza reglas de negocio, autenticacion, roles y acceso a MySQL. El panel web y la app Android consumen la misma API, por lo que los datos creados en un cliente se reflejan en los demas.

La app Android incorpora SQLite como soporte local para mejorar la experiencia y preparar escenarios de sincronizacion. MySQL sigue siendo la fuente oficial de datos. Para que un nuevo desarrollador implemente el proyecto, el flujo recomendado es crear la base desde cero, configurar .env, levantar backend, levantar web, configurar ApiConfig.kt y ejecutar Android desde Android Studio.

Antes de pasar a un entorno productivo se deben reemplazar credenciales demo, usar HTTPS, restringir CORS, definir secretos por ambiente y documentar cualquier cambio de API o base de datos.